

Operis

Gestão operacional de clientes, instalações, treinamentos e acompanhamentos

Documentação técnica e de manutenção

Arquitetura · Tecnologias · Banco de dados · Funcionalidades · Fluxos · Instalação · Configuração · Testes · Deploy · Manutenção · Evolução

Versão 1.0 · 24/09/2026

Sumário

1. Visão geral	4
Problema que resolve	4
Escopo entregue	4
2. Tecnologias utilizadas	5
Decisões técnicas e motivos	5
3. Arquitetura	7
Camadas do servidor (server/src)	7
Estrutura do cliente (client/src)	7
Padrões adotados	7
4. Banco de dados	9
Tabelas e campos principais	9
Categorias de status (kind)	9
Regras de integridade	10
5. Funcionalidades	11
5.1 Dashboard	11
5.2 Clientes	11
5.3 Kanbans (instalações, treinamentos, acompanhamentos, tarefas)	11
5.4 Agendamento e conflitos	11
5.5 Agenda central	11
5.6 Hoje e Atrasados	11
5.7 Conclusão e histórico	12
5.8 Notificações	12
5.9 Pesquisa global	12
5.10 Relatórios e produtividade	12
5.11 Configurações	12
5.12 Usuários e permissões	12
5.13 Interface, acessibilidade e responsividade	13
6. Fluxos do usuário	14
Fluxo principal de um cliente	14
Roteiros rápidos	14
7. Instalação e execução local	15
Requisitos	15
Passo a passo	15

Scripts disponíveis	15
8. Configuração (variáveis de ambiente)	16
Configurações dentro do sistema	16
9. Banco de dados: configuração e inicialização	17
10. Deploy (produção)	18
Opção A — servidor próprio / VPS	18
Opção B — Docker	18
Lista de verificação de produção	18
11. Segurança	19
12. Testes	20
Testes de integração da API (npm test)	20
Roteiro E2E em navegador real (Playwright/Chromium)	20
Responsividade	20
Como testar manualmente após mudanças	20
13. Referência da API	21
14. Manutenção e evolução do código	22
Como fazer mudanças comuns	22
Alterações de esquema (migrações)	22
Convenções	22
Limitações conhecidas	22
15. Melhorias futuras	23
Encerramento	23

1. Visão geral

O **Operis** é um sistema web de gestão operacional para quem atende clientes em ciclos de **instalação → treinamento → acompanhamento**. Ele substitui planilhas, anotações e aplicativos soltos por um único lugar onde se enxerga, em segundos: o que fazer hoje, o que está agendado, o que está atrasado, quem precisa de contato e o que já foi realizado.

Problema que resolve

- **Visibilidade:** dashboard com indicadores, agenda resumida, alertas e área "Atrasados" em destaque.
- **Organização:** cada cliente tem cadastro, jornada, observações rápidas e histórico completo e automático.
- **Execução:** quadros Kanban (instalações, treinamentos, acompanhamentos e tarefas) com arrastar e soltar, agenda diária/semanal/mensal e detecção de conflitos de horário.
- **Prestação de contas:** relatórios por dia/semana/mês/período (PDF, Excel, CSV) e visão de produtividade.
- **Flexibilidade:** status, tipos de treinamento e categorias de tarefa totalmente personalizáveis; nenhum fluxo é obrigatório.

Escopo entregue

Todos os módulos pedidos no escopo original foram implementados: clientes (com favoritos e observações), instalações, treinamentos (com tipos), acompanhamentos, tarefas com checklist, agenda central, dashboard, Hoje/Atrasados, notificações (no sistema e do navegador), pesquisa global (Ctrl+K), filtros por data em todos os módulos, relatórios com exportação, produtividade, configurações e multiusuário com níveis de acesso (administrador/colaborador).

Credenciais iniciais: ao subir com banco vazio é criado o administrador definido em ADMIN_EMAIL / ADMIN_PASSWORD (padrão de desenvolvimento: admin@operis.local / admin123). **Troque a senha antes de usar em produção.** Com `npm run seed:reset` o banco é recriado com dados de demonstração (Restaurante Central, Lanchonete Avenida, Mercado São José, Loja Exemplo e outros) e um colaborador (carla@operis.local / carla123).

2. Tecnologias utilizadas

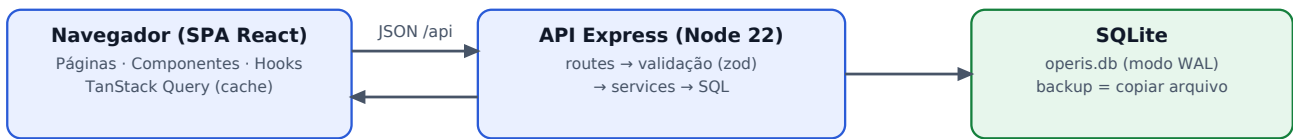
Tecnologia	Onde	Função
Node.js 22	Servidor	Ambiente de execução. Usa <code>process.loadEnvFile</code> para ler o <code>.env</code> sem dependência extra.
TypeScript (strict)	Front e back	Tipagem estática em todo o código; os tipos do front espelham as respostas da API.
Express 5	Servidor	API REST. Erros lançados em handlers síncronos chegam ao middleware de erro automaticamente.
better-sqlite3	Servidor	Banco SQLite embutido (arquivo único), síncrono e rápido, com transações. Não exige instalar servidor de banco.
zod 4	Servidor	Validação de toda entrada (body e query) com mensagens em português para usuários não técnicos.
bcryptjs + jsonwebtoken	Servidor	Hash de senhas e sessão via JWT em cookie <code>httpOnly</code> (7 dias).
helmet, cors, express-rate-limit	Servidor	Cabeçalhos de segurança/CSP, CORS restrito e limite de tentativas de login.
pdfkit / exceljs	Servidor	Geração dos relatórios em PDF profissional e em Excel (3 abas).
React 19 + Vite 7	Cliente	Interface em componentes; Vite para desenvolvimento rápido e build otimizado com divisão de código por página.
React Router	Cliente	Rotas e navegação; páginas secundárias carregadas sob demanda (lazy).
TanStack Query	Cliente	Cache, revalidação e atualização otimista (ex.: mover card no Kanban).
date-fns (pt-BR)	Cliente	Datas e calendário.
lucide-react	Cliente	Ícones (SVG). Status usam ícone + texto + cor (acessibilidade).
CSS próprio com variáveis	Cliente	Design system leve (tokens, modo escuro automático, responsivo). Sem framework de UI, para manter o bundle pequeno.
node:test + Playwright	Testes	11 testes de integração da API e roteiro E2E em navegador real (desktop/tablet/celular).

Decisões técnicas e motivos

- **SQLite em vez de PostgreSQL:** uso individual/pequena equipe, zero administração, backup = copiar um arquivo. O acesso ao banco está isolado em serviços; migrar para PostgreSQL exige trocar a camada `services/` e `db/`, sem tocar rotas nem telas.
- **Tabela única de atividades:** instalações, treinamentos, acompanhamentos, tarefas, reuniões e eventos compartilham a tabela `activities` (campo `type`). Isso dá agenda, atrasos, conflitos, busca e relatórios unificados sem código duplicado. Campos específicos ficam nulos nos demais tipos.
- **Status com "categoria" (kind):** o usuário cria e renomeia status livremente, mas cada um pertence a uma categoria estável (pendente, agendado, em andamento, atenção, concluído, cancelado). Relatórios, atrasos e alertas usam a categoria, então nunca quebram ao personalizar.
- **Datas derivadas, não duplicadas:** data da instalação/treinamento e último/próximo acompanhamento do cliente são calculados a partir das atividades (subconsultas), evitando dados divergentes.
- **Sessão em cookie httpOnly:** o token JWT nunca fica acessível ao JavaScript (mitiga XSS). `SameSite=Lax` mitiga CSRF; a API só aceita JSON.

- **Fuso horário explícito:** o servidor calcula "hoje" e atrasos no fuso configurado (TZ, padrão America/Sao_Paulo), independente do fuso da máquina.

3. Arquitetura



Em produção a API também serve o front compilado (client/dist), gera PDF/XLSX/CSV e roda os lembretes a cada minuto.

Camadas do servidor (server/src)

Camada	Pasta	Responsabilidade
Entrada	index.ts, app.ts	Inicia servidor, middlewares de segurança, serve o front, agenda o gerador de notificações.
Rotas	routes/	Mapeia HTTP → serviços. Valida entrada com zod, aplica permissões (requireAuth / requireAdmin). Sem regra de negócio.
Validação	validation/schemas.ts	Schemas zod (formatos, campos obrigatórios, horários, e-mail, telefone).
Serviços	services/	Regras de negócio e SQL: clientes, atividades (conflitos, conclusão, histórico), status, notificações, dashboard, busca, relatórios.
Relatórios	reports/	Exportadores PDF, XLSX e CSV a partir do mesmo objeto de relatório.
Banco	db/	Conexão, esquema, dados padrão e seed de demonstração.
Transversais	middleware/, utils/	Autenticação, tratamento de erros (sem vaziar detalhes internos), datas/fuso.

Estrutura do cliente (client/src)

Pasta	Conteúdo
pages/	Uma página por rota: Dashboard, Kanban (genérico p/ 4 tipos), Hoje/Atrasados, Agenda, Clientes, Detalhe do cliente, Relatórios, Produtividade, Configurações, Login.
components/ui/	Componentes base reutilizáveis: Button, Modal (foco preso, Esc), Form, Badges, States (vazio/erro/skeleton), Dropdown, PeriodPicker.
components/activities/	Formulário, detalhe, conclusão, card, Kanban, filtros e o <i>provider</i> que abre esses modais de qualquer tela.
components/layout, agenda, charts, clients, settings/	Sidebar/topbar/busca/notificações; grade de horários; gráfico de barras SVG; formulário de cliente; abas de configuração.
hooks/	useAuth, useToast, useConfirm, useLookups (status, usuários...), useDebounce, useMediaQuery.
api/	http.ts (fetch + ApiError) e services.ts (chamadas tipadas agrupadas por módulo).
types/, utils/	Tipos do domínio; datas, formatação, constantes (rótulos, ícones, rotas).

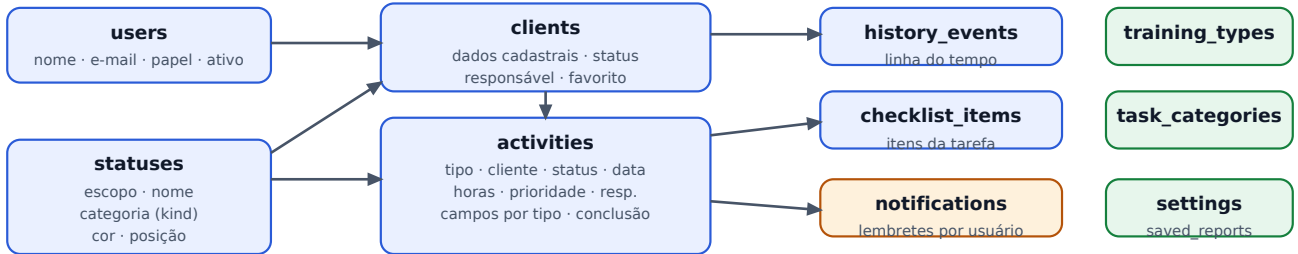
Padrões adotados

- **Um único ponto para abrir modais** (ActivityModalsProvider): qualquer tela chama openDetail, openForm, complete ou move; regras como "concluir pede confirmação" ficam em um só lugar.
- **Kanban genérico**: o mesmo componente atende instalações, treinamentos, acompanhamentos e tarefas; as colunas vêm dos status configurados.

- **Erros padronizados:** a API responde { error, code, fields }; o front mostra a mensagem no campo certo ou em um aviso geral.
- **Ações destrutivas sempre confirmadas** via useConfirm (excluir cliente, atividade, status; cancelar atividade).

4. Banco de dados

SQLite com chaves estrangeiras ativas e modo WAL. O esquema é criado automaticamente (CREATE TABLE IF NOT EXISTS) na primeira execução, junto com os dados padrão (status, tipos de treinamento, categorias, configurações e administrador).



Setas indicam referências (chave estrangeira). training_types e task_categories são referenciadas por activities; settings e saved_reports são independentes.

Tabelas e campos principais

Tabela	Campos principais	Relacionamentos / observações
users	id, name, email (único), password_hash, role (admin member), active	Responsável por clientes e atividades. Nunca excluídos: apenas desativados.
statuses	id, scope (client installation training followup task event), name, kind, color, position	Único por (scope, name). kind ∈ pending, scheduled, in_progress, attention, done, canceled. Ordem = colunas do Kanban.
training_types	id, name (único), active	Referenciado por activities.training_type_id. Em uso → apenas desativa.
task_categories	id, name (único), active	Referenciado por activities.category_id.
clients	name, trade_name, contact_name, phone, whatsapp, email, address, city, notes, status_id, assignee_id, is_favorite, created_at	status_id → statuses; assignee_id → users. Datas de instalação/treinamento/acompanhamento são derivadas.
activities	type, client_id, title, description, status_id, priority, assignee_id, date, start_time, end_time, location, notes; treinamento: training_type_id, content; acompanhamento: contact_reason, situation, problems, solutions, next_followup_date; tarefa: category_id; conclusão: completed_at, completed_date, completed_by, completion_note	client_id → clients (exclusão em cascata). Índices por (type, date), cliente, status, (responsável, data).
checklist_items	activity_id, text, done, position	Exclusão em cascata com a atividade.
history_events	client_id, activity_id, user_id, event_type, description, details (JSON), occurred_at	Registrada automaticamente em cadastro, agendamento, reagendamento, mudança de status, conclusão, cancelamento, exclusão e anotações.
notifications	user_id, kind, title, message, activity_id, client_id, dedupe_key, read_at	UNIQUE(user_id, dedupe_key) garante que cada lembrete seja criado uma única vez.
settings	key, value	Pares chave/valor: dias sem acompanhamento, antecedência de lembrete, duração padrão, empresa...
saved_reports	title, period_from, period_to, generated_by, snapshot (JSON)	Relatórios salvos pelo usuário (resumo do momento).

Categorias de status (kind)

kind	Significado	Efeito no sistema
pending	Ainda não iniciado	Conta como pendente; pode ficar atrasado se tiver data vencida.
scheduled	Data e horário definidos	Exige data. Entra em conflitos de horário e na agenda.
in_progress	Em execução	Conta como aberto.
attention	Precisa de retorno/atenção	Aberto; destaque em alertas.
done	Concluído	Grava data/hora/usuário/observação; alimenta histórico e relatórios (realizado).
canceled	Cancelado / inativo	Sai de conflitos, atrasos e da taxa de conclusão.

Regras de integridade

- Um status em uso só pode ser excluído escolhendo outro status de substituição; sempre deve existir ao menos um status "Concluído" por módulo e ao menos um status por módulo.
- Conflito de horário: mesmo responsável, mesmo dia, intervalos que se sobrepõem (atividades concluídas/canceladas são ignoradas; horários adjacentes não conflitam). O usuário pode confirmar "agendar mesmo assim".
- Cliente duplicado (mesmo nome+cidade ou mesmo telefone/WhatsApp) gera aviso com opção de cadastrar mesmo assim.
- Excluir cliente remove suas atividades e histórico (com confirmação; apenas administrador).
- Reabrir uma atividade concluída limpa data/hora/usuário da conclusão.

5. Funcionalidades

5.1 Dashboard

Indicadores clicáveis (atrasadas, para hoje, clientes, concluídas na semana; instalações, treinamentos e acompanhamentos com pendentes/agendados/em andamento/concluídos), lista **Atrasados** em destaque vermelho, **Hoje** em ordem cronológica, **Próximos 7 dias** com filtro por tipo, **Alertas** (atraso, compromisso de hoje, instalação/treinamento próximo, cliente sem acompanhamento), clientes que precisam de acompanhamento (com botão "Agendar") e clientes importantes.

5.2 Clientes

- Cadastro completo com máscara de telefone, validação de e-mail/telefone e detecção de duplicidade; cadastro rápido também dentro do formulário de atividades.
- Lista com busca, filtros (status, cidade, responsável, importantes), ordenação por coluna, paginação; tabela no desktop e cartões no celular.
- **Detalhe:** jornada (Cadastro → Instalação → Treinamento → Acompanhamentos → Conclusão), atividades em aberto, todas as atividades, dados, **observações rápidas** editáveis no local, histórico completo em linha do tempo, anotações no histórico, ligar/WhatsApp/e-mail e atalhos para criar atividades.
- Favoritos (★) aparecem na área de acesso rápido do dashboard.

5.3 Kanbans (instalações, treinamentos, acompanhamentos, tarefas)

- Colunas = status configurados. Arrastar e soltar altera o status automaticamente; mover para "Concluído" abre o diálogo de conclusão; para "Cancelado" pede confirmação; para "Agendado" sem data abre a edição.
- Card com cliente, data, horário, responsável, cidade, status, observações, prioridade (cor + texto), progresso do checklist e destaque de atrasado. Menu do card: mover para..., concluir, editar, excluir.
- No celular: abas de coluna + rolagem por coluna (scroll-snap) e menu "Mover para" (arrastar não é usado em toque).
- Filtros: busca, período (hoje, amanhã, esta semana, próxima semana, este mês, mês anterior, personalizado), situação (inclui "Atrasadas"), prioridade, cidade, responsável.

5.4 Agendamento e conflitos

Formulário único adaptado ao tipo: instalação (local pré-preenchido com o endereço do cliente), treinamento (tipo e conteúdo), acompanhamento (motivo, situação, problemas, soluções, próximo acompanhamento), tarefa (categoria, descrição, checklist), reunião/evento. O horário final é sugerido a partir da duração padrão. Ao salvar, a API valida e informa conflitos de horário com o detalhamento do compromisso existente.

5.5 Agenda central

Visões **dia** e **semana** (grade horária com eventos sobrepostos lado a lado, linha do "agora" e faixa "sem hora"), e **mês** (chips no desktop, pontos coloridos no celular + lista do dia selecionado). Clicar em um evento abre os detalhes; clicar em um horário/dia vazio cria um evento. No celular a semana vira uma lista por dia. Filtros por tipo.

5.6 Hoje e Atrasados

Hoje: tudo do dia em ordem cronológica, contadores (a fazer, concluídos, horário vencido) e concluídos recolhíveis. **Atrasados:** tudo que passou da data/horário e não foi concluído, agrupado por data, com filtros por tipo/cidade/responsável. O menu lateral exibe a quantidade de atrasados.

5.7 Conclusão e histórico

Concluir registra automaticamente data, horário, usuário, status e observação. Para acompanhamentos é possível já informar o **próximo acompanhamento** (o sistema cria o próximo); após instalação/treinamento é possível agendar o primeiro acompanhamento. Tudo alimenta o histórico do cliente e os relatórios.

5.8 Notificações

Sino com contador e painel: *Hoje*, *Em breve* (dentro da antecedência configurada), *Amanhã*, *Atrasado* e *Cliente sem acompanhamento*. Gerado no servidor a cada minuto, sem duplicidade. Notificações do navegador (Notification API) para itens novos enquanto o sistema está aberto, mediante permissão.

5.9 Pesquisa global

Ctrl/⌘+K em qualquer tela. Pesquisa clientes (nome, fantasia, telefone, cidade, e-mail), atividades (título, cliente, cidade, responsável, status; também por tipo — "instalação" — e por data — "25/09") e histórico. Resultados agrupados, navegáveis por teclado.

5.10 Relatórios e produtividade

- Período: dia, semana, mês ou personalizado, com navegação anterior/próximo; botão **Gerar relatório da semana**.
- Conteúdo: indicadores, resumo por tipo (total, concluídas, pendentes, agendadas, em andamento, canceladas, atrasadas), clientes (novos, atendidos, ativos, finalizados), tempo em atividades concluídas, lista de atividades e observações importantes.
- Exportação: **PDF** (cabeçalho, cartões-resumo, tabelas, paginação), **Excel** (abas Resumo, Atividades, Observações) e **CSV** (UTF-8 com BOM, separador ";" para abrir no Excel em português). Relatórios podem ser salvos.
- Produtividade: taxa de conclusão, concluídas por dia (barras), instalações/treinamentos/acompanhamentos por semana (barras empilhadas), atrasadas e tempo. Gráficos têm descrição acessível, tooltip e alternativa em tabela.
- Regra de período: atividades concluídas contam pela data de conclusão; as demais pela data prevista (ou de criação, se sem data).

5.11 Configurações

Dados pessoais e senha; usuários (criar, editar, papel, ativar/desativar); status por módulo (criar, editar, cor, categoria, reordenar, excluir com substituição); tipos de treinamento; categorias de tarefa; preferências (empresa, duração padrão, antecedência de lembretes, dias para alertar falta de acompanhamento, notificações do navegador).

5.12 Usuários e permissões

Ação	Administrador	Colaborador
Ver/editar atividades	Todas	Somente as atribuídas a ele (outras retornam 403)
Clientes	Ver, criar, editar, excluir	Ver, criar, editar (sem excluir)

Ação	Administrador	Colaborador
Status, tipos, categorias, usuários, configurações gerais	Sim	Não (somente leitura das listas)
Dashboard e relatórios	Visão completa	Somente suas atividades

5.13 Interface, acessibilidade e responsividade

- Visual SaaS/CRM: sidebar, cards, badges, modo escuro automático (prefers-color-scheme), microanimações discretas (desligadas com prefers-reduced-motion).
- Status e prioridade sempre com **ícone + texto + cor**. Foco visível, modais com foco preso, rótulos ARIA, tabelas com aria-sort.
- Desktop: sidebar fixa. Tablet: grades reflow. Celular: menu em gaveta, barra inferior com botão "+" central, modais em folha inferior, listas em cartões, Kanban por colunas, agenda em lista.
- Estados vazios com ação ("Nenhuma instalação encontrada. + Nova instalação"), skeletons de carregamento, toasts de sucesso/erro e mensagens de validação claras.

6. Fluxos do usuário

Fluxo principal de um cliente

Cliente cadastrado

- ↳ Instalação (Pendente → Agendada → Em andamento → Concluída)
 - ↳ Treinamento (Pendente → Agendado → Em andamento → Concluído)
 - ↳ Acompanhamento 1 (Acompanhar → Agendado → Em contato → Realizado)
 - ↳ Acompanhamento 2 ... n (cada conclusão pode criar o próximo)
 - ↳ Cliente "Finalizado"

Nenhuma etapa é obrigatória: qualquer atividade pode ser criada em qualquer ordem.

Roteiros rápidos

Objetivo	Passos
Cadastrar cliente	+ Novo → Cliente (ou página Clientes → Novo cliente) → nome obrigatório → Salvar. Abre o detalhe do cliente.
Agendar instalação	Detalhe do cliente → + Instalação (ou Instalações → + Nova) → data e horário → Salvar. Se houver conflito, ajustar ou "Agendar mesmo assim".
Alterar status	Arrastar o card no Kanban; ou menu ... → Mover para; ou detalhe → Alterar status.
Concluir	Botão Concluir (detalhe/menu) ou arrastar para a coluna Concluído → observação opcional → Concluir.
Registrar acompanhamento	+ Acompanhamento → preencher motivo/situação/problemas/soluções → Concluir informando o próximo acompanhamento.
Ver o dia	Menu Hoje (cronológico) ou Agenda → Dia. Atrasados no menu lateral.
Gerar relatório	Relatórios → Gerar relatório da semana (ou escolher período) → PDF / Excel / CSV.
Encontrar algo	Ctrl+K → digitar nome, telefone, cidade, "instalação" ou "25/09".

7. Instalação e execução local

Requisitos

- Node.js 22.13 ou superior e npm 10+ (necessário para carregar .env e para o better-sqlite3 pré-compilado).
- Nenhum banco de dados externo.

Passo a passo

```
# 1) instalar dependências (raiz, servidor e cliente)
npm install          # instala concurrently na raiz
npm run setup        # instala server/ e client/

# 2) configurar variáveis de ambiente
cp server/.env.example server/.env    # edite os valores

# 3) (opcional) dados de demonstração
npm run seed:reset

# 4) desenvolvimento: API em :3333 e front em :5173 (proxy /api)
npm run dev
# abra http://localhost:5173
```

Scripts disponíveis

Comando	O que faz
npm run dev	API (tsx watch) + Vite com recarregamento automático.
npm run build	Compila o front (client/dist) e o servidor (server/dist).
npm start	Executa o servidor compilado; serve API e front na mesma porta.
npm run seed	Insere dados de demonstração se o banco estiver sem clientes.
npm run seed:reset	APAGA o banco e recria com dados de demonstração.
npm test	Testes de integração da API (11 testes, banco em memória).
npm run typecheck	Verificação de tipos do servidor e do cliente.

8. Configuração (variáveis de ambiente)

Arquivo server/.env (modelo em .env.example). Nunca versione o .env.

Variável	Padrão	Descrição
PORT	3333	Porta HTTP.
NODE_ENV	development	"production" ativa validações rígidas (JWT_SECRET obrigatório) e cookie Secure.
JWT_SECRET	(dev)	Obrigatório em produção , mínimo 24 caracteres aleatórios. Trocar invalida todas as sessões.
DATABASE_PATH	./data/operis.db	Arquivo SQLite (a pasta é criada automaticamente).
TZ	America/Sao_Paulo	Fuso usado para "hoje", atrasos e data de conclusão.
CLIENT_ORIGIN	http://localhost:5173	Origem permitida no CORS (apenas desenvolvimento; em produção o front é servido pela própria API).
ADMIN_NAME / ADMIN_EMAIL / ADMIN_PASSWORD	Administrador / admin@operis.local / admin123	Administrador criado no primeiro start, apenas se não houver usuários.
COOKIE_SECURE	true em produção	Defina "false" somente se ainda não usa HTTPS (ex.: teste em rede local).

Configurações dentro do sistema

Em Configurações → Preferências: nome da empresa (relatórios), duração padrão, antecedência de lembretes, dias para alertar cliente sem acompanhamento e notificações do navegador. Status, tipos e categorias têm abas próprias.

9. Banco de dados: configuração e inicialização

- O arquivo é criado em `DATABASE_PATH` na primeira execução, com o esquema, status padrão, tipos de treinamento, categorias, configurações e o administrador inicial.
- Não há ferramenta de migração externa: o esquema usa `CREATE TABLE IF NOT EXISTS`. Para alterações futuras, veja a seção de manutenção.
- **Backup:** com o servidor parado, copie `operis.db` (e os arquivos `-wal/-shm`, se existirem). Com o servidor ativo use `sqlite3 operis.db ".backup backup.db"`. Agende cópias diárias.
- **Restaurar:** pare o servidor, substitua o arquivo e inicie novamente.
- **Reset de demonstração:** `npm run seed:reset` (apaga tudo).

10. Deploy (produção)

Opção A — servidor próprio / VPS

```
git clone <repositorio> operis && cd operis
npm install && npm run setup
cp server/.env.example server/.env # define NODE_ENV=production, JWT_SECRET forte, ADMIN_*, DATABASE_PATH
npm run build
NODE_ENV=production npm start      # ou use PM2/systemd:
# pm2 start "npm start" --name operis && pm2 save
```

Coloque um proxy reverso com HTTPS (Nginx, Caddy ou Traefik) apontando para a porta 3333.

Exemplo com Caddy: `operis.seudominio.com { reverse_proxy localhost:3333 }`. Com HTTPS o cookie de sessão é marcado como Secure automaticamente.

Opção B — Docker

```
# arquivo .env na raiz: JWT_SECRET=... ADMIN_PASSWORD=... COOKIE_SECURE=true (atrás de HTTPS)
docker compose up -d --build
# dados persistem no volume "operis-data" (/data/operis.db)
```

Lista de verificação de produção

- JWT_SECRET longo e aleatório; senha do administrador trocada; HTTPS ativo.
- Backup diário do arquivo do banco.
- TZ correto para o fuso do negócio.
- Limite de login já ativo (20 tentativas / 15 min em produção).
- Se a equipe crescer muito ou for necessário mais de um servidor, migrar para PostgreSQL (ver melhorias).

11. Segurança

Tema	Como é tratado
Senhas	Hash bcrypt (custo 10); nunca retornadas pela API.
Sessão	JWT em cookie httpOnly + SameSite=Lax (+ Secure em produção), 7 dias; o usuário é revalidado no banco a cada requisição (usuário desativado perde acesso na hora).
Autorização	requireAuth em todas as rotas exceto login/health; requireAdmin nas ações administrativas; colaborador filtrado por responsável no nível do serviço.
Entrada	Todos os corpos e consultas validados com zod; SQL sempre parametrizado (sem concatenar valores do usuário); limite de 200 KB por requisição.
Cabeçalhos	helmet com CSP restritiva (scripts apenas do próprio domínio), x-powered-by desligado; CORS restrito à origem configurada.
Força bruta	express-rate-limit no login.
Erros	Resposta 500 genérica; detalhes apenas no log do servidor. Nenhum segredo é enviado ao front.
Front	React escapa conteúdo por padrão; não há uso de innerHTML; token inacessível ao JavaScript.

12. Testes

Testes de integração da API (npm test)

11 testes com banco SQLite em memória e servidor real, cobrindo: autenticação; cliente (cadastrar, validar, duplicidade, visualizar, editar, excluir); instalação (criar, agendar, conflito, horário adjacente, forçar, status, concluir, histórico); Kanban (mudar/reabrir e status de outro módulo); acompanhamento com próximo acompanhamento; agenda e filtros de data/atrasados; tarefa com checklist; status personalizados (criar, duplicado, exclusão em uso com substituição, proteção do último "Concluído"); relatórios e exportação PDF/XLSX/CSV; dashboard, busca e notificações; permissões de colaborador.

Roteiro E2E em navegador real (Playwright/Chromium)

Executado contra o sistema em execução, com banco de demonstração: login inválido e válido; validação de campos; cadastro de cliente e redirecionamento; histórico; agendamento com horário final sugerido; **detecção de conflito** e "agendar mesmo assim"; **arrastar e soltar** entre colunas; conclusão via Kanban com observação; menu "Mover para"; checklist; busca global agrupada; criação/edição/exclusão de evento pela agenda (com confirmação); download de PDF e Excel; salvar relatório; gráficos de produtividade; criação de status e tipo de treinamento; exclusão de cliente com confirmação. Resultado: **24 verificações aprovadas**, sem erros de JavaScript.

Responsividade

Capturas e verificação automática de estouro horizontal em 1440×900 (desktop), 820×1100 (tablet) e 390×844 (celular) para todas as 13 rotas: nenhum estouro; menu em gaveta, barra inferior, formulários em folha inferior e Kanban por colunas validados visualmente.

Como testar manualmente após mudanças

- `npm run typecheck` e `npm test`.
- Rodar `npm run seed:reset` && `npm run dev` e percorrer: criar cliente → instalação → arrastar → concluir → relatório.
- Redimensionar a janela (ou usar o modo dispositivo do navegador) para 390 px.

13. Referência da API

Base: /api. Autenticação por cookie. Erros: { "error": "mensagem", "code": "CODIGO", "fields": { "campo": "mensagem" } }. Códigos úteis: VALIDATION, DUPLICATE_CLIENT, SCHEDULE_CONFLICT (409; reenviar com force: true para confirmar), STATUS_IN_USE (409; reenviar com ?replace_with=ID).

Método e rota	Descrição
POST /auth/login · POST /auth/logout · GET/PUT /auth/me	Sessão e dados pessoais (troca de senha exige senha atual).
GET /dashboard	Indicadores, listas do dia/atrasados/próximos, alertas, favoritos.
GET /search?q=	Clientes, atividades e histórico.
GET/POST /clients · GET/PUT/DELETE /clients/:id	CRUD (filtros q, status_id, city, assignee_id, favorite, sort, order, page). Detalhe inclui atividades e histórico. DELETE só admin.
PATCH /clients/:id/favorite · POST /clients/:id/notes · GET /clients/cities	Favorito, anotação no histórico, lista de cidades.
GET/POST /activities · GET/PUT/DELETE /activities/:id	CRUD. Filtros: type, status_id, kind, client_id, assignee_id, city, priority, range (today tomorrow week next_week month last_month custom), from, to, q, overdue, sort.
PATCH /activities/:id/status · POST /activities/:id/complete	Mudar status (Kanban) e concluir (nota, próximo acompanhamento, status de conclusão).
PATCH /checklist/:itemId · POST /activities/check-conflicts	Marcar item; consultar conflitos de horário.
GET/POST/PUT/DELETE /statuses · PUT /statuses/reorder	Status por módulo (escrita apenas admin).
/training-types · /task-categories	CRUD de apoio (escrita admin). Em uso: desativa em vez de excluir.
GET/POST/PUT /users · GET/PUT /settings	Usuários e configurações (escrita admin).
GET /notifications · POST /notifications/read-all · POST /notifications/:id/read	Central de notificações.
GET /reports · POST /reports/save · GET /reports/saved · GET /reports/export/(pdf xlsx csv)	Relatório por período (range/from/to) e exportações.

14. Manutenção e evolução do código

Como fazer mudanças comuns

Mudança	Onde mexer
Novo campo em cliente	schema.ts (coluna) → validation/schemas.ts (clientSchema) → services/clients.ts (lista F e SELECT) → types/index.ts → ClientForm.tsx e ClientDetailPage.tsx.
Novo campo em atividade	schema.ts → COLS em services/activities.ts → activitySchema → types/index.ts (Activity) → ActivityForm.tsx / ActivityDetail.tsx.
Novo tipo de atividade	CHECK de type em schema.ts, ActivityType em types (server e client), ACTIVITY_META em constants.ts, scopeOf, status padrão em DEFAULT_STATUSES; para Kanban próprio, rota em KANBAN_ROUTES e item no menu.
Novo status padrão	DEFAULT_STATUSES em db/schema.ts (só vale para bancos novos; em bancos existentes use Configurações → Status).
Nova regra de negócio	Sempre em services/ (nunca em rotas ou componentes); cobrir em test/api.test.ts.
Novo relatório/exportação	services/reports.ts (dados) + reports/ (formato).
Nova tela	pages/ + rota em App.tsx + item em AppLayout (NAV); reutilize components/ui e ActivityModals.

Alterações de esquema (migrações)

O esquema atual é criado por `CREATE TABLE IF NOT EXISTS`, que não altera tabelas existentes. Para evoluir um banco já em uso: (1) faça backup; (2) adicione a coluna ao schema.ts (para bancos novos); (3) aplique um `ALTER TABLE ... ADD COLUMN` no startup de db/connection.ts protegido por verificação (`PRAGMA table_info`) ou adote uma tabela `schema_migrations` com scripts numerados. Recomenda-se adotar migrações numeradas antes da primeira mudança de esquema em produção.

Convenções

- TypeScript estrito; sem any desnecessário nas telas; SQL parametrizado.
- Mensagens ao usuário em português claro; mensagens de erro de validação definidas no zod.
- Um componente = uma responsabilidade; lógica compartilhada em hooks/serviços.
- Datas: strings ISO (AAAA-MM-DD) e HH:MM trafegam entre camadas; formatação BR apenas na exibição.
- Não altere status "kind" existentes sem revisar relatórios (services/reports.ts) e alertas (dashboard/notifications).

Limitações conhecidas

- SQLite: uma única instância do servidor (adequado para uma equipe pequena).
- Notificações do navegador funcionam com o sistema aberto (não há push em segundo plano).
- A ordem dos cards dentro de uma coluna é automática (prioridade, depois data); não há ordenação manual.
- Lista de clientes no seletor de atividades carrega os 200 primeiros por nome (suficiente para o uso atual; trocar por busca assíncrona se crescer).
- Sem migrações versionadas de banco (ver acima).

15. Melhorias futuras

- **WhatsApp:** botão já abre conversa (wa.me); evoluir para envio automático de lembretes/confirmações via API oficial (o gerador de notificações em `services/notifications.ts` é o ponto de extensão).
- **Google Calendar:** sincronização bidirecional de atividades com data/hora (cada atividade já tem início/fim e local).
- **Notificações push / PWA:** service worker + Web Push; manifesto para instalar como app no celular; modo offline básico.
- **Aplicativo mobile:** a API REST já é independente do front; um app nativo/React Native pode reutilizar tudo.
- **IA:** resumos automáticos do histórico do cliente, sugestão de próximos acompanhamentos e texto do relatório semanal.
- **Multiempresa:** adicionar `organization_id` às tabelas e filtrar nos serviços (o acesso a dados já está centralizado).
- **Permissões avançadas:** papéis adicionais e permissões por módulo (hoje admin/colaborador, aplicados em `requireAdmin` e no filtro de responsável).
- **Dashboard avançado:** metas, comparativos entre períodos, funil de clientes, tempo médio entre etapas.
- **Ordenação manual dos cards,** ações em lote, anexos (fotos/termos de instalação), assinatura digital do cliente, campos personalizados.
- **Infra:** migrações versionadas, PostgreSQL, logs estruturados, testes de componentes (Vitest) e E2E no CI, auditoria de acesso.

Encerramento

O projeto foi entregue com revisão completa: tipagem estrita sem erros, testes automatizados aprovados, roteiro E2E sem erros de console, verificação de responsividade nos três tamanhos e build de produção validado. Este documento deve ser atualizado junto com mudanças de arquitetura ou de banco.